Linear Regression

Aldebaro Klautau
Federal University of Pará (UFPA) / LASSE

PPGEE – UFPA - Brazil

Dec., 2024

Linear Regression

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- Bias or intercept
- Recall the equation of a line:
  y = mx + b

- $\hat{y}$ is the predicted value.
- n is the number of features.
- x_i is the ith feature value.
- θ_j is the jth model parameter (including the bias term θ_0 and the feature weights $\theta_1, \theta_2, \cdots, \theta_n$).

Vectorized form:

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$$

- $\boldsymbol{\theta}$ is the model's *parameter vector*, containing the bias term θ_0 and the feature weights θ_1 to θ_n.
- $\mathbf{x}$ is the instance's *feature vector*, containing x_0 to x_n, with x_0 always equal to 1.
- $\boldsymbol{\theta} \cdot \mathbf{x}$ is the dot product of the vectors $\boldsymbol{\theta}$ and $\mathbf{x}$, which is of course equal to $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$.
- $h_{\boldsymbol{\theta}}$ is the hypothesis function, using the model parameters $\boldsymbol{\theta}$.

Simple linear algebra when m = 2 examples

Express the simultaneous equations

$$3a + b = 13 \quad ✓$$

$$2a + b = 7 \quad ✓$$

as a matrix equation.

$$\boxed{\begin{bmatrix} 3 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 13 \\ 7 \end{bmatrix}}$$

coefficient matrix — variable matrix — constant matrix

$$ax + by = c$$

In our ML case:

- **a** is the θ_1 parameter that multiplies input x
- **b** is the bias coeficiente θ_0, that is always multiplied by one
- **c** is the correct label
- and we have m > 2 training examples, such that the system is overdetermined (more equations than variables)

NOTE

In Machine Learning, vectors are often represented as *column vectors*, which are 2D arrays with a single column. If $\boldsymbol{\theta}$ and $\mathbf{x}$ are column vectors, then the prediction is $\hat{y} = \boldsymbol{\theta}^\intercal \mathbf{x}$, where $\boldsymbol{\theta}^\intercal$ is the *transpose* of $\boldsymbol{\theta}$ (a row vector instead of a column vector) and $\boldsymbol{\theta}^\intercal \mathbf{x}$ is the matrix multiplication of $\boldsymbol{\theta}^\intercal$ and $\mathbf{x}$. It is of course the same prediction, except that it is now represented as a single-cell matrix rather than a scalar value. In this book I will use this notation to avoid switching between dot products and matrix multiplications.

Training the linear regression model

- Most common performance measure of a regression model is the Root Mean Square Error (RMSE). Therefore, to train a Linear Regression model, we need to find the value of θ that minimizes the RMSE. In practice, it is simpler to minimize the mean squared error (MSE) than the RMSE, and it leads to the same result (because the value that minimizes a function also minimizes its square root)

$$\mathrm{MSE}\left(\mathbf{X}, h_{\boldsymbol{\theta}}\right) = \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^{\mathsf{T}} \mathbf{x}^{(i)} - y^{(i)}\right)^2$$

Normal equation

$$\widehat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal \mathbf{X})^{-1}\, \mathbf{X}^\intercal\, \mathbf{y}$$

- $\widehat{\boldsymbol{\theta}}$ is the value of $\boldsymbol{\theta}$ that minimizes the cost function.
- $\mathbf{y}$ is the vector of target values containing $y^{(1)}$ to $y^{(m)}$.

$\widehat{\boldsymbol{\theta}} = \mathbf{X}^{+}\mathbf{y}$, where $\mathbf{X}^{+}$ is the *pseudoinverse* of $\mathbf{X}$

Specifically, the Moore-Penrose inverse. You can use np.linalg.pinv() to compute the pseudoinverse directly

Advanced material

For those who really want to learn about pseudinverses

A.15.4 Moore-Penrose pseudoinverse

Pseudoinverses[2] are generalizations of the inverse matrix and are useful when the given matrix does not have an inverse (for example, when the matrix is not square or full rank).

The Moore-Penrose pseudoinverse has several interesting properties and is adequate to least square problems. It provides the minimum-norm least squares solution $\mathbf{z} = \mathbf{X}\mathbf{b}$ to the problem of finding a vector $\mathbf{z}$ that minimizes the error vector norm $||\mathbf{X}\mathbf{z} - \mathbf{b}||$. Assuming $\mathbf{X}$ is an $m \times n$ matrix, the pseudoinverse provides the solution for a set of overdetermined or underdetermined equations if $m > n$ or $m < n$, respectively.

Two properties of a Moore-Penrose pseudoinverse $\mathbf{X}^+$ are

$$\mathbf{X}^H = \mathbf{X}^H \mathbf{X} \mathbf{X}^+, \tag{A.35}$$

and

$$\mathbf{X}^H = \mathbf{X}^+ \mathbf{X} \mathbf{X}^H. \tag{A.36}$$

With r being the rank of $\mathbf{X}$, then:

- if $m = n$ and $r = m = n$, the pseudoinverse $\mathbf{X}^{+} = \mathbf{X}^{-1}$ is equivalent to the usual inverse;
- if $m > n$ (overdetermined) and, besides, $r = n$ (the columns of $\mathbf{X}$ are linearly independent), $\mathbf{X}^{H}\,\mathbf{X}$ is invertible and using Eq. (A.35) the pseudoinverse is given by

$$\mathbf{X}^{+} = \left(\mathbf{X}^{H}\,\mathbf{X}\right)^{-1}\mathbf{X}^{H}; \tag{A.37}$$

- if $n > m$ (underdetermined) and $r = m$ (the rows of $\mathbf{X}$ are linearly independent), $\mathbf{X}\,\mathbf{X}^{H}$ is invertible and using Eq. (A.36) leads to

$$\mathbf{X}^{+} = \mathbf{X}^{H}\left(\mathbf{X}\,\mathbf{X}^{H}\right)^{-1}. \tag{A.38}$$

Whenever available, instead of Eq. (A.37) or Eq. (A.38) that requires linear independence, one should use a robust method to obtain $\mathbf{X}^{+}$ such as the pinv function in Matlab/Octave, which adopts a SVD decomposition to calculate $\mathbf{X}^{+}$. Listing A.3 illustrates such calculations and the convenience of relying on pinv when the independence of rows or columns is not guaranteed.

From https://github.com/aldebaro/dsp-telecom-book-code/blob/master/MatlabOctaveCodeSnippets/snip_systems_pseudo_inverse.m

```
test=3; %choose the case below
switch test
    case 1 %m>n (overdetermined/tall) and linearly indepen. columns
        X=[1 2 3; -4+1j -5+1j -6+1j;1 0 0;0 1 0]; %4 x 3
    case 2 %n>m (underdetermined/fat) and linearly independent rows
        X=[1 2 3; -4+1j -5+1j -6+1j]; %2 x 3
    case 3 %neither rows nor columns of X are linearly independent
        %rows X(2,:)=2*X(1,:) and columns X(:,4)=3*X(:,1)
        X=[1 2 3 3; 2 4 6 6; -4+1j -5+1j -6+1j -12+3*1j]; %3 x 4
end
Xp_svd = pinv(X) %pseudoinverse via SVD decomposition
Xp_over = inv(X'*X)*X' %valid when columns are linearly independent
Xp_under = X'*inv(X*X') %valid when rows are linearly independent
rank(X'*X) %X'*X is square but may not be full rank
rank(X*X') %X*X' is square but may not be full rank
Xhermitian=X'*X*pinv(X) %equal to X' (this property is always valid)
Xhermitian2=pinv(X)*X*X' %equal to X' (the property itself is valid)
maxError_over=max(abs(Xp_svd(:)-Xp_over(:))) %error for overdetermin.
maxError_under=max(abs(Xp_svd(:)-Xp_under(:))) %for underdetermined
```

localhost:8888/notebooks/04_training_linear_models.ipynb#
Personal life | LASSE | Work - others | Deep Learning | Python | Communications | Pagamentos | Samurai papers | CurrentWork | Família | GoogleDrive | Other bookmarks
jupyter 04_training_linear_models Last Checkpoint: 09/26/2021 (autosaved)
Logout
File Edit View Insert Cell Kernel Widgets Help
Trusted
Python (ai6genv)
Run
Code

The Normal Equation

In [2]:
```
import numpy as np

X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
```

In [3]:
```
plt.plot(X, y, "b.")
plt.xlabel("$x_1$", fontsize=18)
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.axis([0, 2, 0, 15])
save_fig("generated_data_plot")
plt.show()
```

```
Saving figure generated_data_plot
```

Computational Complexity

The Normal Equation computes the inverse of $\mathbf{X}^\intercal \mathbf{X}$, which is an $(n + 1) \times (n + 1)$ matrix (where n is the number of features). The *computational complexity* of inverting such a matrix is typically about $O(n^{2.4})$ to $O(n^3)$, depending on the implementation. In other words, if you double the number of features, you multiply the computation time by roughly $2^{2.4} = 5.3$ to $2^3 = 8$.

The SVD approach used by Scikit-Learn's `LinearRegression` class is about $O(n^2)$. If you double the number of features, you multiply the computation time by roughly 4.

WARNING

Both the Normal Equation and the SVD approach get very slow when the number of features grows large (e.g., 100,000). On the positive side, both are linear with regard to the number of instances in the training set (they are $O(m)$), so they handle large training sets efficiently, provided they can fit in memory.